INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DA PARAÍBA
UNIDADE ACADÊMICA DE INFORMAÇÃO E COMUNICAÇÃO
COORDENAÇÃO DO CURSO SUPERIOR DE TECNOLOGIA EM SISTEMAS PARA INTERNET

RELATÓRIO DE ESTÁGIO

Desenvolvimento Full-Stack de um Módulo de Solicitações para Transição de Categoria no Sistema Banco de Especialistas

Luis Kilmer da Silva Bernardo

João Pessoa - PB
10/2026

Instituto Federal de Educação, Ciência e Tecnologia da Paraíba
Unidade Acadêmica de Informação e Comunicação
Coordenação do Curso Superior de Tecnologia em Sistemas para Internet

Desenvolvimento Full-Stack de um Módulo de Solicitações para Transição de Categoria no Sistema Banco de Especialistas

Luis Kilmer da Silva Bernardo

Relatório de Estágio apresentado ao Coordenação do Curso Superior de Tecnologia em Sistemas para Internet da Instituto Federal de Educação, Ciência e Tecnologia da Paraíba, como requisito parcial para a obtenção do título de Tecnólogo.

Orientador:	Juliana Dantas Ribeiro Viana de Medeiros
Supervisor:	Juliana Dantas Ribeiro Viana de Medeiros
Coordenador do Curso:	Cândido José Ramos do Egypto
Empresa:	Agência de Inovação (NEO-IFPB)
Período:	25/02/2026 à 19/06/2026

João Pessoa - PB

APROVAÇÃO

Esta folha deverá ser substituída pela cópia digitalizada da folha de aprovação fornecida.

Juliana Dantas Ribeiro Viana de Medeiros
Supervisor da Empresa

Cândido José Ramos do Egypto
Coordenador do CST de Sistemas para Internet

Prof. Dr. Juliana Dantas Ribeiro Viana de Medeiros
Professor Orientador

Luis Kilmer da Silva Bernardo
Estagiário

Altere este texto inserindo a dedicatória do seu trabalho.

AGRADECIMENTOS

Edite e coloque aqui os agradecimentos às pessoas e/ou instituições que contribuíram para a realização do trabalho.

É obrigatório o agradecimento às instituições de fomento à pesquisa que financiaram total ou parcialmente o trabalho, inclusive no que diz respeito à concessão de bolsas.

Eu denomino meu campo de Gestão do Conhecimento, mas você não pode gerenciar conhecimento. Ninguém pode. O que pode fazer - o que a empresa pode fazer - é gerenciar o ambiente que otimize o conhecimento. (PRUSAK, Laurence, 1997).

RESUMO

Silva Bernardo, Luis Kilmer. Desenvolvimento Full-Stack de um Módulo de Solicitações para Transição de Categoria no Sistema Banco de Especialistas. 10/2026. 19 f. Relatório de Estágio – Coordenação do Curso Superior de Tecnologia em Sistemas para Internet, Instituto Federal de Educação, Ciência e Tecnologia da Paraíba. João Pessoa - PB, 10/2026.

Este relatório apresenta as atividades desenvolvidas no Laboratório ASSERT do Instituto Federal da Paraíba (IFPB), Campus João Pessoa, entre fevereiro e junho de 2026, por meio do PIAGI. O estágio teve como objetivo desenvolver um módulo para o sistema Banco de Especialistas, visando formalizar e rastrear solicitações de alterações cadastrais e de classificação dos usuários. Para isso, implementou-se o módulo de Solicitações, contemplando a modelagem das entidades Solicitação e Snapshot, as regras de negócio para criação, edição, aprovação e rejeição de solicitações e uma linha do tempo (timeline) para registrar os eventos e as decisões do processo. O desenvolvimento também abrangeu a implementação de operações de persistência e endpoints, a validação e serialização de dados, a integração com a arquitetura existente e a construção de funcionalidades de interface para solicitantes e avaliadores. A solução utilizou Python, Flask, PostgreSQL, SQLAlchemy e Marshmallow no back-end, Nuxt no front-end e Docker e Docker Compose para configuração do ambiente. Também foram desenvolvidos testes automatizados com Pytest para verificar as funcionalidades implementadas. Como resultado, o módulo integrou o fluxo de solicitações à plataforma, reduzindo a dependência de comunicações por e-mail e intervenções manuais no banco de dados, além de proporcionar maior formalização, rastreabilidade e histórico das alterações realizadas.

Palavras-chave: Desenvolvimento full-stack. Banco de Especialistas. Solicitações. Rastreabilidade. Testes automatizados.

LISTA DE FIGURAS

LISTA DE QUADROS

LISTA DE TABELAS

Tabela 1 – Códigos HTTP e suas aplicações no módulo 4

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
ACID	Atomicidade, Consistência, Isolamento e Durabilidade (Atomicity, Consistency, Isolation, Durability)
CNPq	Conselho Nacional de Desenvolvimento Científico e Tecnológico
DECOM	Departamento de Computação
IFPB	Instituto Federal da Paraíba
JSON	Notação de Objetos JavaScript (JavaScript Object Notation)
JSONB	Notação de Objetos JavaScript Binário (JavaScript Object Notation Binary)
MVCC	Controle de Concorrência Multiversão (Multiversion Concurrency Control)
ORM	Mapeamento Objeto-Relacional (Object-Relational Mapping)
PIAGI	Programa Institucional de Apoio à Gestão da Inovação
REST	Transferência de Estado Representacional (Representational State Transfer)
SGBD	Sistema Gerenciador de Banco de Dados
SQL	Linguagem de Consulta Estruturada (Structured Query Language)
URI	Identificador Uniforme de Recurso (Uniform Resource Identifier)
WSGI	Interface de Gateway de Servidor Web (Web Server Gateway Interface)

LISTA DE SÍMBOLOS

Γ	Letra grega Gama
λ	Comprimento de onda
$\in$	Pertence

LISTA DE ALGORITMOS

SUMÁRIO

LISTA DE FIGURAS viii

LISTA DE QUADROS ix

LISTA DE TABELAS x

LISTA DE ALGORITMOS xiii

1 – INTRODUÇÃO 1

1.1 Objetivos 1

1.1.1 Objetivo geral 1

1.1.2 Objetivos específicos 2

1.2 Escopo e limitações 2

1.3 Procedimentos metodológicos 2

1.4 Contexto institucional 3

1.5 Organização do relatório 3

2 – EMBASAMENTO TÉORICO 4

2.1 Arquitetura Web Cliente-Servidor 4

2.2 Framework Flask 5

2.3 Persistência de dados com PostgreSQL e SQLAlchemy 6

2.3.1 PostgreSQL 6

2.3.2 SQLAlchemy 7

2.4 Serialização e Validação de Dados com Marshmallow 7

2.5 Auditoria, Rastreabilidade e Versionamento de Dados 8

2.5.1 Auditoria e rastreabilidade 8

2.5.2 Fluxo de aprovação com máquina de estados 8

2.5.3 Snapshot e reversão 8

2.5.4 Timeline e registro de eventos 9

3 – ATIVIDADES REALIZADAS 10

4 – CONSIDERAÇÕES FINAIS 11

Referências 12

Apêndices 14

APÊNDICE A–Nome do apêndice 15

APÊNDICE B–Nome do outro apêndice 16

Anexos 17

ANEXO A–Nome do anexo 18

ANEXO B–Nome do outro anexo 19

1 INTRODUÇÃO

O presente relatório descreve as atividades desenvolvidas durante o estágio supervisionado realizado entre fevereiro e junho de 2026 no Laboratório ASSERT, vinculado ao Polo de Inovação do Instituto Federal da Paraíba (IFPB). As atividades concentraram-se na evolução do sistema Banco de Especialistas, plataforma destinada ao gerenciamento de especialistas e ao apoio à composição de equipes para projetos de Pesquisa, Desenvolvimento e Inovação (PD&I).

O Banco de Especialistas permite o cadastro, a classificação e a avaliação de candidatos com base em critérios como formação acadêmica, experiência profissional e participação em projetos. Nesse contexto, a plataforma reúne informações utilizadas na gestão dos especialistas e nos processos de seleção e composição de equipes vinculadas às iniciativas do Polo de Inovação.

Durante o estágio, identificou-se uma demanda relacionada à alteração de categoria dos especialistas. O procedimento então utilizado exigia que o interessado solicitasse a mudança por e-mail e que um administrador realizasse manualmente uma consulta SQL diretamente no banco de dados. A ausência de um fluxo formal de solicitações dificultava o acompanhamento das alterações e a recuperação do estado anterior dos registros, além de aumentar o risco de inconsistências decorrentes de intervenções diretas na base de dados.

Diante desse cenário, o trabalho concentrou-se no desenvolvimento de um módulo de solicitações para formalizar e gerenciar operações de alteração cadastral e de classificação dos especialistas. A solução contempla mecanismos para registrar solicitações, acompanhar seu andamento, permitir sua análise e decisão e manter informações necessárias à rastreabilidade das operações. Dessa forma, busca-se reduzir a dependência de intervenções manuais no banco de dados e estabelecer um processo mais controlado e verificável.

Do ponto de vista técnico, o desenvolvimento envolveu conceitos de engenharia de software, aplicações web, persistência de dados relacionais, testes automatizados e conteinerização. Destacam-se a integração entre componentes de front-end e back-end, o uso de mecanismos de registro e recuperação do estado dos usuários e a manutenção de um histórico das operações. Esses fundamentos são apresentados no capítulo dedicado ao embasamento teórico.

1.1 Objetivos

1.1.1 Objetivo geral

Desenvolver um módulo de solicitações de alteração cadastral e de classificação no sistema Banco de Especialistas, contemplando os fluxos de criação, consulta, aprovação, rejeição e reversão, com mecanismos de rastreabilidade e registro das operações.

1.1.2 Objetivos específicos

- Levantar e analisar os requisitos e as regras de negócio relacionados às solicitações de alteração cadastral e de classificação dos especialistas.
- Modelar os componentes de persistência necessários ao registro das solicitações e à recuperação do estado dos usuários.
- Implementar as funcionalidades de gerenciamento das solicitações no back-end, incluindo as regras de negócio, a integração com o PostgreSQL e o acesso aos dados por meio do SQLAlchemy.
- Desenvolver as interfaces web necessárias à criação, ao acompanhamento e à análise das solicitações.
- Implementar mecanismos de rastreabilidade das operações, incluindo o registro de eventos e a apresentação de seu histórico.
- Desenvolver e executar testes unitários e de integração para verificar as regras de negócio e o funcionamento integrado dos componentes da aplicação.

1.2 Escopo e limitações

O trabalho abrange o desenvolvimento do módulo de solicitações e as funcionalidades necessárias ao gerenciamento das operações de alteração cadastral e de classificação dos especialistas no sistema Banco de Especialistas. Estão incluídos os mecanismos de registro, acompanhamento, decisão e reversão das solicitações, bem como os recursos de persistência e rastreabilidade associados a esse fluxo.

Não fazem parte do escopo a redefinição dos critérios de pontuação dos especialistas, a modificação do processo seletivo, a substituição do sistema de avaliação existente e a implementação de notificações por e-mail. Também não é contemplado o tratamento de múltiplas solicitações simultâneas para um mesmo especialista. Essas limitações delimitam as responsabilidades da solução desenvolvida e podem orientar evoluções futuras do sistema.

1.3 Procedimentos metodológicos

O trabalho caracteriza-se como um relato técnico de natureza aplicada e caráter descritivo, desenvolvido no contexto de um estágio supervisionado em desenvolvimento de software. Sua execução compreendeu o estudo da arquitetura e das regras de negócio do sistema, o levantamento e a análise dos requisitos, a modelagem dos componentes necessários, a implementação das funcionalidades e a realização de testes.

A verificação técnica envolveu testes unitários e de integração, destinados a avaliar o comportamento das funcionalidades implementadas e a interação entre os componentes da aplicação. Também foi verificado o funcionamento integrado do front-end e do back-end em ambiente de desenvolvimento conteinerizado com Docker. As atividades foram conduzidas no

contexto da arquitetura e das tecnologias já adotadas pelo projeto, considerando suas restrições e regras de negócio.

1.4 Contexto institucional

O estágio foi realizado no âmbito do Polo de Inovação do Instituto Federal da Paraíba, estrutura associada à Agência de Inovação da instituição (NEO-IFPB). O Polo atua no desenvolvimento de iniciativas de Pesquisa, Desenvolvimento e Inovação (PD&I), aproximando a instituição de demandas tecnológicas do setor produtivo e da sociedade.

As atividades técnicas foram desenvolvidas no Laboratório ASSERT, ambiente vinculado ao Polo de Inovação no qual são realizadas atividades de desenvolvimento e manutenção de sistemas computacionais para projetos institucionais. Nesse contexto, o estágio proporcionou a participação nas etapas de análise, implementação, integração e verificação de funcionalidades de uma aplicação web existente.

1.5 Organização do relatório

Além deste capítulo introdutório, o relatório está organizado em três capítulos. O segundo apresenta o embasamento teórico relacionado às tecnologias e aos conceitos utilizados no desenvolvimento. O terceiro descreve as atividades realizadas durante o estágio, incluindo a análise dos requisitos, a implementação das funcionalidades e os testes executados. Por fim, o quarto capítulo apresenta as considerações finais, sintetizando os resultados do trabalho, as limitações identificadas e os possíveis desdobramentos.

2 EMBASAMENTO TÉORICO

Este capítulo apresenta, de forma clara, os conceitos e as tecnologias que fundamentam as atividades descritas neste relatório. Como o estágio teve por foco o desenvolvimento de um módulo de solicitações com a rastreabilidade, auditoria e reversão para o sistema Banco de Especialistas, é dada ênfase ao *back-end* (Flask, SQLAlchemy e PostgreSQL) e aos conceitos de auditoria e versionamento de dados. As demais tecnologias (Vue.js, Nuxt, Pytest, Git e Docker) são abordadas como elementos de apoio ao desenvolvimento, à validação e à execução do módulo.

2.1 Arquitetura Web Cliente-Servidor

A arquitetura cliente-servidor organiza um sistema distribuído em dois papéis complementares: o cliente, que solicita serviços, e o servidor, que os provê. Na Web, o cliente é tipicamente um navegador que envia requisições por meio do protocolo HTTP, e o servidor processa essas requisições, aplica as regras de negócio, acessa a camada de persistência e devolve uma resposta (TANENBAUM; van Steen, 2007).

O estilo arquitetural REST foi definido por Fielding (2000) como um conjunto de restrições para sistemas hipermídia distribuídos. Entre elas destacam-se a interface uniforme, a ausência de estado entre requisições e a identificação de recursos por meio de URIs. Em uma API REST, os recursos do domínio (por exemplo, uma solicitação de alteração de categoria) são manipulados por meio dos métodos HTTP.

O HTTP é um protocolo sem estado (stateless): cada requisição é independente das anteriores e deve carregar as informações necessárias ao seu processamento, como credenciais ou tokens de autenticação.

Tabela 1 – Códigos HTTP, significados padrão e aplicações no módulo.

Código HTTP	**Significado padrão**	**Aplicação no módulo**
200	Sucesso	Consultas e operações concluídas com êxito.
201	Recurso criado	Criação de uma nova solicitação.
400	Requisição inválida	Erros de validação nos dados enviados.
401	Falha de autenticação	Sem permissão adequada de acesso.
403	Proibido	Tentativa de acesso a recurso sem permissão.
404	Não encontrado	Solicitação ou recurso inexistente.
409	Conflito de estado	Tentativa de transição inválida (por exemplo, reverter solicitação não aprovada).

Fonte: Elaboração própria, com base em RFC 9110 (FIELDING; NOTTINGHAM; RESCHKE, 2022).

As respostas utilizam códigos de status que expressam o resultado da operação, conforme

detalhado na Tabela 1. O formato de troca de dados utilizado é o JSON, definido pela (ECMA INTERNATIONAL, 2013) como uma sintaxe leve, baseada em texto e independente de linguagem para intercâmbio de dados. Ele é nativamente suportado pelas tecnologias usadas no projeto, como JavaScript e Python.

Uma característica relevante dessa arquitetura é a separação de responsabilidades. A camada de apresentação, executada no cliente, concentra-se na interação com o usuário; a camada de aplicação, no servidor, concentra as regras de negócio; e a camada de dados garante o armazenamento e a recuperação das informações. Essa divisão favorece a manutenção e a evolução independente de cada parte, além de permitir que diferentes clientes consumam o mesmo servidor. É uma boa prática manter uma aplicação seguindo esse formato até para reutilização de recursos já projetos e boilerplate de outros projetos.

O Banco de Especialistas adota esse modelo: uma interface web, construída com Vue.js e Nuxt, comunica-se com uma API desenvolvida em Flask, que por sua vez persiste os dados em um banco PostgreSQL. O módulo de solicitações foi desenvolvido sobre essa estrutura, incorporando regras de negócio no servidor, de modo que a validade das operações não dependa do comportamento do cliente.

2.2 Framework Flask

O Flask é um microframework para desenvolvimento web em Python, criado por Armin Ronacher. É considerado "micro"porque seu núcleo é enxuto e não impõe escolhas sobre camada de persistência, validação ou autenticação, deixando essas decisões a cargo do desenvolvedor, que pode incorporar extensões conforme a necessidade (GRINBERG, 2018). Sua implementação baseia-se em duas bibliotecas: o Werkzeug, que fornece a camada de comunicação conforme a especificação WSGI (Web Server Gateway Interface), e o Jinja2, mecanismo de templates (PALLETS, 2024).

O mecanismo central do Flask é o roteamento: as funções Python (as views) são associadas a rotas e métodos HTTP por meio de decoradores. Cada função recebe a requisição, executa a lógica necessária e retorna uma resposta. Outros recursos relevantes incluem:

- Blueprints: permitem organizar a aplicação em módulos independentes, cada um com suas rotas. Esse recurso é adequado para isolar o módulo de solicitações do restante do sistema.
- Contextos de aplicação e de requisição: disponibilizam, durante o tratamento de uma requisição, objetos como a própria requisição e a configuração da aplicação, sem necessidade de passá-los explicitamente entre funções.
- Tratamento de erros: possibilita registrar manipuladores que convertem exceções do domínio em respostas HTTP padronizadas.
- Fábrica de aplicação (application factory): padrão em que a aplicação é criada por uma função, o que facilita a configuração distinta para ambientes de desenvolvimento, produção e testes.

A flexibilidade do Flask justifica sua adoção em projetos nos quais a equipe deseja controlar a composição da arquitetura. Em contrapartida, exige disciplina na organização do código, em especial na separação entre rotas, regras de negócio e acesso a dados, para evitar views extensas e de difícil teste. Em um módulo com regras como mudança de categoria e recálculo de pontuação, essa separação é particularmente importante.

2.3 Persistência de dados com PostgreSQL e SQLAlchemy

A persistência de dados é definida como a característica que garante que as informações manipuladas por um sistema sobrevivam à execução do processo que as criou, sendo armazenadas permanentemente em meios não voláteis (SILBERSCHATZ; KORTH; SUDARSHAN, 2020). No contexto de sistemas web, como o módulo desenvolvido, a persistência é o mecanismo que assegura que as solicitações, as alterações de categoria e os registros de auditoria permaneçam disponíveis e íntegros, independentemente do encerramento da aplicação ou de falhas no servidor.

Para concretizar essa persistência e garantir a integridade e a rastreabilidade estrutural das informações, o projeto utiliza o PostgreSQL como Sistema Gerenciador de Banco de Dados (SGBD) e o SQLAlchemy como ferramenta de abstração e mapeamento objeto-relacional, tecnologias detalhadas a seguir.

2.3.1 PostgreSQL

O PostgreSQL é um sistema gerenciador de banco de dados objeto-relacional de código aberto, reconhecido por sua aderência ao padrão SQL, extensibilidade e robustez (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2024). Como SGBD relacional, organiza os dados em tabelas relacionadas por chaves primárias e estrangeiras, garantindo integridade referencial.

Um conceito central para o módulo desenvolvido é o de transação. Segundo Elmasri e Navathe (ELMASRI; NAVATHE, 2011), uma transação é uma unidade lógica de trabalho que deve respeitar as propriedades ACID: atomicidade (todas as operações são efetivadas ou nenhuma é), consistência (a transação leva o banco de um estado válido a outro), isolamento (transações concorrentes não interferem indevidamente entre si) e durabilidade (dados confirmados persistem mesmo diante de falhas). O PostgreSQL implementa o controle de concorrência por meio de MVCC (Multiversion Concurrency Control), o que permite leituras e escritas simultâneas com bom desempenho.

A atomicidade é crítica em operações como a aprovação de uma solicitação, que envolve alterar a categoria do especialista, atualizar sua pontuação, registrar o estado anterior e atualizar o status da solicitação. Se qualquer uma dessas etapas falhar, todas devem ser desfeitas, evitando inconsistências entre categoria, pontuação e histórico.

2.3.2 SQLAlchemy

O SQLAlchemy é uma biblioteca Python para acesso a bancos de dados que oferece tanto uma camada de abstração SQL (Core) quanto um mapeador objeto-relacional (ORM) (BAYER, 2012). O ORM resolve o chamado descompasso de impedância entre o modelo orientado a objetos e o modelo relacional: tabelas são mapeadas para classes, linhas para instâncias e colunas para atributos, de modo que consultas e alterações podem ser expressas em código Python, sem a escrita direta de SQL na maior parte dos casos.

O SQLAlchemy implementa padrões descritos por Fowler (FOWLER, 2006), em especial o Data Mapper, que separa os objetos de domínio do mecanismo de persistência, e o Unit of Work, que acompanha as alterações feitas nos objetos durante uma sessão e as grava no banco de forma coordenada, dentro de uma única transação. Na prática, o objeto Session representa essa unidade de trabalho: alterações são acumuladas e efetivadas por commit, ou descartadas por rollback.

Outros recursos relevantes são a definição de relacionamentos entre entidades (um-para-muitos, muitos-para-muitos), o carregamento tardio (lazy loading) ou antecipado (eager loading) de dados relacionados, e o uso de consultas parametrizadas, que previnem ataques de injeção de SQL. Esse último ponto contrasta diretamente com o processo anterior ao módulo, em que o administrador executava consultas SQL manuais sobre os registros, sem validações estruturadas ou registro sistêmico das alterações.

2.4 Serialização e Validação de Dados com Marshmallow

O Marshmallow é uma biblioteca Python para serialização, desserialização e validação de dados (MARSHMALLOW DEVELOPERS, 2024). A serialização converte objetos da aplicação (como instâncias de modelos ORM) em tipos simples, que podem ser transformados em JSON e enviados ao cliente. A desserialização realiza o caminho inverso: recebe dados externos, valida-os e os converte em estruturas utilizáveis pela aplicação.

O elemento central é o *schema*, uma classe que declara os campos esperados, seus tipos e suas regras de validação, como obrigatoriedade, tamanho máximo, formato ou valores permitidos. Validações adicionais, específicas do domínio, podem ser implementadas por funções de validação. Quando os dados são inválidos, a biblioteca produz mensagens de erro estruturadas, que podem ser devolvidas ao cliente com código HTTP 400.

O uso de schemas na fronteira da API cumpre duas funções. A primeira é de segurança e robustez: dados externos nunca devem ser considerados confiáveis, e a validação sistemática impede que entradas malformadas alcancem as regras de negócio. A segunda é de padronização: os schemas definem de maneira explícita e única o formato das requisições e respostas, servindo como contrato entre o front-end e o back-end. Essa característica é relevante quando as interfaces de criação, acompanhamento e análise de solicitações dependem de formatos de dados estáveis.

2.5 Auditoria, Rastreabilidade e Versionamento de Dados

Esta seção reúne os conceitos que sustentam a principal contribuição do módulo: a capacidade de registrar, consultar e desfazer alterações cadastrais com garantias de rastreabilidade.

2.5.1 Auditoria e rastreabilidade

Auditoria, no contexto de sistemas de informação, é o registro sistemático de quem realizou determinada operação, quando, sobre qual dado e com qual resultado. A rastreabilidade é a propriedade que permite reconstituir o histórico de uma informação a partir desses registros. Fowler (FOWLER, 2004) descreve o padrão *Audit Log* como um registro de alterações feitas nos dados, com o objetivo de permitir verificar o que ocorreu e, quando necessário, compreender a origem de um estado atual. Em sistemas que envolvem decisões de classificação e seleção de pessoas, como o Banco de Especialistas, esse registro contribui para a transparência e para a responsabilização dos agentes envolvidos.

O fluxo anterior ao módulo, baseado em e-mail e execução manual de SQL, ilustra a ausência dessas propriedades: não havia registro formal do solicitante, da justificativa, do responsável pela decisão ou do estado anterior do cadastro. Ao formalizar a solicitação como uma entidade persistente, com status, autoria e datas, o sistema passa a produzir esse registro como efeito natural do próprio fluxo de trabalho.

2.5.2 Fluxo de aprovação com máquina de estados

Uma solicitação percorre um ciclo de vida bem definido, que pode ser modelado como uma máquina de estados finitos: um conjunto de estados (por exemplo, pendente, aprovada, rejeitada e revertida) e de transições permitidas entre eles (SOMMERVILLE, 2011). Explicitar as transições válidas evita estados incoerentes: uma solicitação rejeitada não pode ser aprovada sem nova análise, e somente uma solicitação aprovada pode ser revertida. A validação dessas regras no servidor garante que o fluxo seja respeitado independentemente da interface utilizada.

2.5.3 Snapshot e reversão

Para possibilitar a reversão de uma alteração, é necessário preservar o estado anterior dos dados. O conceito de *snapshot* (instantâneo) consiste em capturar e armazenar o estado de um objeto em determinado momento. Ele se aproxima do padrão *Memento*, descrito por Gamma et al. (GAMMA et al., 1994), cujo propósito é capturar o estado interno de um objeto, sem violar seu encapsulamento, para que ele possa ser restaurado posteriormente.

No módulo, a entidade *Snapshot* registra o estado relevante do especialista (como categoria e pontuação) antes da aplicação de uma alteração aprovada. A reversão consiste em restaurar esses valores, mantendo a consistência entre categoria e pontuação. Cabe distinguir essa abordagem da simples exclusão do registro de histórico: a reversão é, ela mesma, uma

operação registrada, de modo que o histórico permanece completo e a operação inversa pode ser auditada. Esse comportamento é análogo à ideia de transações compensatórias, em que o efeito de uma operação já efetivada é desfeito por uma nova operação, e não pela remoção do passado.

Existem abordagens mais gerais para o versionamento de dados, como o *Event Sourcing*, em que o estado é derivado de uma sequência imutável de eventos (FOWLER, 2005; KLEPPMANN, 2017). O módulo adota uma solução mais simples e proporcional ao escopo, combinando *snapshots* do estado anterior com um registro de eventos, e o conceito de *Event Sourcing* é aqui mencionado como referência conceitual para a ideia de histórico imutável.

2.5.4 Timeline e registro de eventos

A *timeline* é a apresentação ordenada cronologicamente dos eventos associados a uma entidade. Cada evento (criação, aprovação, rejeição, reversão) é registrado com seu tipo, autor, data e informações complementares, e a interface os exibe como uma linha do tempo. Trata-se de uma forma de tornar a auditoria acessível ao usuário final, e não apenas ao administrador do banco de dados.

3 ATIVIDADES REALIZADAS

Cada capítulo deve conter uma pequena introdução (tipicamente, um ou dois parágrafos) que deve deixar claro o objetivo e o que será discutido no capítulo, bem como a organização do capítulo.

4 CONSIDERAÇÕES FINAIS

Cada capítulo deve conter uma pequena introdução (tipicamente, um ou dois parágrafos) que deve deixar claro o objetivo e o que será discutido no capítulo, bem como a organização do capítulo.

Referências

BAYER, M. Sqlalchemy. In: BROWN, A.; WILSON, G. (Ed.). **The Architecture of Open Source Applications, Volume II: Structure, Scale, and a Few More Fearless Hacks**. Lulu.com, 2012. Disponível em: <https://aosabook.org/en/v2/sqlalchemy.html>. Acesso em: 25 out. 2026. Citado na página 7.

ECMA INTERNATIONAL. **ECMA-404: The JSON Data Interchange Syntax**. 1. ed. Genebra, Suíça, 2013. Disponível em: <https://www.ecma-international.org/publications/files/ECMA-ST/ECMA-404.pdf>. Acesso em: 25 out. 2026. Citado na página 5.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**. 6. ed. São Paulo: Pearson Addison Wesley, 2011. Citado na página 6.

FIELDING, R. T.; NOTTINGHAM, M.; RESCHKE, J. **HTTP Semantics**. [S.l.], 2022. Disponível em: <https://www.rfc-editor.org/info/rfc9110>. Acesso em: 25 out. 2026. Citado na página 4.

FOWLER, M. **Audit Log**. 2004. Martin Fowler's Bliki. Disponível em: <https://martinfowler.com/eaaDev/AuditLog.html>. Acesso em: 25 out. 2026. Citado na página 8.

FOWLER, M. **Event Sourcing**. 2005. Martin Fowler's Bliki. Disponível em: <https://martinfowler.com/eaaDev/EventSourcing.html>. Acesso em: 25 out. 2026. Citado na página 9.

FOWLER, M. **Padrões de Arquitetura de Aplicações Corporativas**. Porto Alegre: Bookman, 2006. Obra original publicada em 2002. Citado na página 7.

GAMMA, E. et al. **Design Patterns: Elements of Reusable Object-Oriented Software**. Reading, Massachusetts: Addison-Wesley, 1994. Citado na página 8.

GRINBERG, M. **Flask Web Development: Developing Web Applications with Python**. 2. ed. Sebastopol, CA: O'Reilly Media, 2018. Citado na página 5.

KLEPPMANN, M. **Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems**. Sebastopol, CA: O'Reilly Media, 2017. Citado na página 9.

MARSHMALLOW DEVELOPERS. **marshmallow: simplified object serialization**. [S.l.], 2024. Disponível em: <https://marshmallow.readthedocs.io/>. Acesso em: 25 out. 2026. Citado na página 7.

PALLETS. **Flask Documentation**. [S.l.], 2024. Disponível em: <https://flask.palletsprojects.com/>. Acesso em: 25 out. 2026. Citado na página 5.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 16 Documentation**. [S.l.], 2024. Disponível em: <https://www.postgresql.org/docs/16/>. Acesso em: 25 out. 2026. Citado na página 6.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. **Sistema de Banco de Dados**. 7. ed. Rio de Janeiro: Gen LTC, 2020. Citado na página 6.

SOMMERVILLE, I. **Engenharia de Software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011. Citado na página 8.

TANENBAUM, A. S.; van Steen, M. **Sistemas Distribuídos: Princípios e Paradigmas**. 2. ed. São Paulo: Pearson Prentice Hall, 2007. Citado na página 4.

Apêndices

APÊNDICE A – Nome do apêndice

Lembre-se que a diferença entre apêndice e anexo diz respeito à autoria do texto e/ou material ali colocado.

Caso o material ou texto suplementar ou complementar seja de sua autoria, então ele deverá ser colocado como um apêndice. Porém, caso a autoria seja de terceiros, então o material ou texto deverá ser colocado como anexo.

Caso seja conveniente, podem ser criados outros apêndices para o seu trabalho acadêmico. Basta recortar e colar este trecho neste mesmo documento. Lembre-se de alterar o "label" do apêndice.

Não é aconselhável colocar tudo que é complementar em um único apêndice. Organize os apêndices de modo que, em cada um deles, haja um único tipo de conteúdo. Isso facilita a leitura e compreensão para o leitor do trabalho.

APÊNDICE B – Nome do outro apêndice

conteúdo do novo apêndice

Anexos

ANEXO A – Nome do anexo

Lembre-se que a diferença entre apêndice e anexo diz respeito à autoria do texto e/ou material ali colocado.

Caso o material ou texto suplementar ou complementar seja de sua autoria, então ele deverá ser colocado como um apêndice. Porém, caso a autoria seja de terceiros, então o material ou texto deverá ser colocado como anexo.

Caso seja conveniente, podem ser criados outros anexos para o seu trabalho acadêmico. Basta recortar e colar este trecho neste mesmo documento. Lembre-se de alterar o "label" do anexo.

Organize seus anexos de modo a que, em cada um deles, haja um único tipo de conteúdo. Isso facilita a leitura e compreensão para o leitor do trabalho. É para ele que você escreve.

ANEXO B – Nome do outro anexo

conteúdo do outro anexo